

Plano de Refatoração do Backend em 3 Meses

Projeto: Fiestou

Responsável técnico: Pedro (Tech Lead / Desenvolvedor Principal)

Posicionamento do documento: escrevi este plano para execução real, com foco em reduzir risco de negócio durante a transição de backend. Não é texto acadêmico; é um plano de execução com ordem clara, critérios de aceite e rollback definidos.

MVP em produção

Migração progressiva

Zero downtime como alvo

Paridade funcional obrigatória

Sumário

- Contexto e objetivo de negócio
- Princípios técnicos da migração
- Escopo do remake e fronteiras
- Arquitetura alvo (.NET + RavenDB) em nível prático
- Plano de execução por semana (12 semanas)
- Contrato de dados e migração MySQL → RavenDB
- Plano de APIs, versionamento e compatibilidade
- Pagamentos, antecipações e reconciliação
- Qualidade, testes e critérios de go/no-go
- Observabilidade, segurança e operação
- Cutover, rollback e continuidade
- Riscos, mitigação e decisões pendentes
- Orçamento de esforço e distribuição semanal
- Anexos operacionais (checklists e runbooks)
- Análises específicas e melhorias priorizadas para produção

1) Contexto e objetivo de negócio

O objetivo principal do remake não é “trocar stack por trocar stack”. O objetivo é garantir continuidade de crescimento da Fiestou com backend mais previsível para escalar produto, operação e financeiro sem aumentar fragilidade da plataforma.

Hoje já temos backend funcional em Laravel/PHP com MySQL e muita regra de negócio consolidada em produção. Isso significa que a refatoração precisa respeitar o que já funciona e preservar comportamento, especialmente em:

- Checkout e fechamento de pedido
- Split de pagamento e liquidação por loja
- Painel do lojista (pedidos e financeiro)
- Painel admin (visão completa e operação)
- Fluxos críticos de status de pedido e entrega

Regra de ouro: nenhuma melhoria arquitetural justifica quebrar fluxo de receita. O roadmap técnico deve andar

junto com proteção da conversão e da operação diária.

1.1 Resultado esperado ao fim dos 3 meses

- Nova API .NET cobrindo os fluxos essenciais do backend atual.
- RavenDB rodando como banco principal para domínios migrados.
- Convivência controlada entre legado e novo backend durante transição.
- Cutover com rollback validado e documentado.
- Time apto a operar o novo stack sem dependência de improviso.

1.2 O que não vamos fazer nesse ciclo

- Reescrever frontend completo junto com backend.
- Mudar regra de produto/negócio sem necessidade para migração.
- Introduzir integrações novas externas sem benefício imediato para cutover.
- Refinar modelagem profunda de RavenDB antes da validação técnica final.

2) Princípios técnicos da migração

Vou conduzir essa refatoração com princípios objetivos para evitar refação da refatoração:

Princípio	Aplicação prática	Indicador
Strangler Pattern	Migrar por domínio, mantendo legado vivo temporariamente.	Rota por rota trocando para .NET com feature flag.
Paridade antes de otimização	Primeiro reproduzir comportamento atual, depois otimizar.	Diferença funcional zero nos cenários críticos.
Fonte única de verdade por domínio	Evitar dupla escrita sem governança.	Dono de dados por contexto definido no plano.
Observabilidade nativa	Logs estruturados, trace e métricas desde semana 1.	p95, taxa de erro, eventos de negócio monitorados.
Rollback first	Cada etapa já nasce com rollback desenhado.	Teste de rollback executado antes de cada canary.

2.1 Critérios de início (gate contratual + técnico)

1. MVP estabilizado sem bug crítico em produção por no mínimo 15 dias.
2. Escopo mínimo de funcionalidades essenciais documentado e assinado internamente.
3. Ambientes staging/prod com backup e healthcheck operacionalmente confiáveis.
4. Métricas de baseline do legado registradas (latência, erro, throughput).

2.2 Regras de execução

- Sem push direto sem validação em staging.
- Sem corte em produção sem checklist de go/no-go.
- Sem “atalho” em pagamento, split ou conciliação financeira.
- Toda decisão estrutural relevante vai para ADR curto (1 página).

3) Escopo do remake e fronteiras

O remake em 3 meses não é “100% do sistema em nível de perfeccionismo”. É **100% dos fluxos essenciais com paridade e operação segura**. Isso inclui:

3.1 Domínios que entram

- Autenticação e autorização (base para painéis e APIs sensíveis)
- Catálogo (lojas, produtos, atributos, disponibilidade)
- Carrinho e checkout (incluindo regras de locação e frete)
- Pedidos e status (cliente, lojista, admin)
- Pagamentos e reconciliação
- Financeiro lojista/admin (recebimentos/antecipações/repasses)
- Rotinas operacionais e jobs críticos

3.2 Domínios com profundidade reduzida neste ciclo

- Analytics avançado de recomendação (mantém versão funcional mínima)
- Módulos experimentais/legados de baixo impacto comercial
- Refino de UX (fica no ciclo seguinte, exceto correções críticas)

Direção prática: priorizar o que impacta receita, operação e confiança do parceiro. O resto entra no hardening pós-cutover.

4) Arquitetura alvo (.NET + RavenDB) em nível prático

4.1 Estrutura da solução .NET

```
src/
  Fiestou.Api/           # camada HTTP
  Fiestou.Application/   # casos de uso
  Fiestou.Domain/        # regras de negócio
  Fiestou.Infrastructure/ # RavenDB, gateways, filas, integrações
  Fiestou.Contracts/     # DTOs e contratos versionados
  Fiestou.Jobs/          # workers/schedulers
  tests/
    Fiestou.UnitTests/
    Fiestou.IntegrationTests/
    Fiestou.ContractTests/
```

4.2 Contextos (bounded contexts) iniciais

Contexto	Responsabilidade	Dependências externas
Identity	login, sessão, autorização por perfil	JWT, middleware auth
Catalog	lojas, produtos, variações, mídia, disponibilidade	storage de imagens
Checkout	carrinho, cálculo de totais/frete, criação de pedido	endereços, regras comerciais
Payments	ordens de pagamento, webhook, split, reconciliação	Pagar.me
Orders	timeline, status, visão por perfil	pagamentos, entrega
Finance	saldos, antecipações, repasses, conciliação	Pagar.me, admin panel

4.3 Modelagem inicial em RavenDB (sem exagero antecipado)

Aqui vou manter modelagem objetiva. Sem hiperotimização prematura em RavenDB antes da validação técnica final.

Coleções iniciais

- Users
- Stores
- Products
- Carts
- Orders
- Payments
- FinancialMovements
- AuditEvents

Índices iniciais

- Orders_ByStore_AndStatus
- Orders_ByCustomer_AndDate
- Products_ByStore_AndActive
- Financial_ByStore_AndPeriod
- Payments_ByProviderId

4.4 Pontos pendentes para validação técnica final

Tema	Status	Dono	Prazo	Gate	Impacto
Estratégia final de versionamento de documentos Orders	Aguardando nota técnica	Pedro	Semana 4	Gate Onda 3	Médio/alto no histórico de pedido
Padrão definitivo de consistência transacional entre contextos	Aguardando revisão	Pedro	Semana 8	Gate Onda 4	Alto em checkout/pagamento
Política de sessões e refresh tokens no novo auth	Aguardando validação	Pedro	Semana 4	Gate Onda 3	Alto em segurança
Tuning de índices dinâmicos vs estáticos	Aguardando benchmark conjunto	Pedro	Semana 8	Gate Onda 4	Médio em performance

5) Plano de execução por semana (12 semanas)

Semanas 1-2 | Fundação, inventário e baseline

1. Congelar escopo crítico do ciclo e documentar matriz de paridade.
2. Inventariar endpoints, jobs, webhooks, cron e scripts de operação do legado.
3. Subir solução .NET base com healthcheck, autenticação e log estruturado.
4. Configurar RavenDB em ambiente de staging e pipeline CI/CD inicial.
5. Capturar baseline legado: p95, taxa de erro, throughput e funil de checkout.

Entregáveis semana 2

- Documento de paridade publicado.
- API .NET “up” com rota de health, auth mínima e tracing funcional.
- Runbook de deploy inicial e rollback técnico base.

Semanas 3-4 | Catálogo (read/write) + backfill inicial

1. Migrar leitura de lojas/produtos/atributos para .NET em staging.
2. Migrar escrita de cadastro/edição de produto para .NET (sem alterar UX).
3. Executar backfill MySQL → RavenDB para dados mestres.
4. Criar comparador de paridade de resposta (legado x .NET) para catálogo.
5. Entrar em shadow mode para rotas de catálogo.

Entregáveis semana 4

- Catálogo operando via .NET em staging com paridade validada.
- Relatório de divergência de dados e correções aplicadas.

Semanas 5-6 | Carrinho, checkout e cálculo

1. Portar regras de cálculo de carrinho (subtotal/frete/regras de loja).
2. Portar validações de atributos e quantidades de variações.
3. Criar suíte de testes de cálculo com cenários reais de produção.
4. Integrar criação de pedido em .NET mantendo contrato atual de frontend (pacotes W8 e W9 concluídos nesta fase).

Entregáveis semana 6

- Checkout criando pedido no .NET em staging.
- Comparativo automático de totais legado x .NET sem divergência crítica.

Semanas 7-8 | Pagamento, webhook e reconciliação

1. Implementar camada de pagamento e idempotência por evento (somente após fechamento do W9).
2. Portar webhook de confirmação e atualização de status de pedido.
3. Implementar reconciliação automática e fallback para atrasos de webhook.
4. Proteger contrato de pedidos por perfil (admin/lojista/cliente).
5. Portar endpoints de financeiro necessários para painel.

Entregáveis semana 8

- Fluxo de pagamento completo no .NET em staging.

- Pedidos com status consistente entre evento de pagamento e painel.
- Auditoria mínima persistida (provider_id, status, timestamps).

Semanas 9-10 | Painéis, hardening e paralelo assistido

1. Substituir consumo de dados dos painéis para domínios já migrados.
2. Fechar lacunas de paridade de admin/lojista/cliente.
3. Executar testes de carga e estabilidade em staging.
4. Executar modo paralelo assistido (dual-read) com comparação em tempo real.
5. Treinar operação para incidentes no novo backend.

Entregáveis semana 10

- Painéis consumindo .NET nos fluxos essenciais.
- Relatório de hardening com ajustes de performance e confiabilidade.

Semanas 11-12 | Canary em produção, cutover e encerramento técnico

1. Canary por rota/domínio de baixo risco e depois médio risco.
2. Monitorar métricas de negócio e técnica em tempo real.
3. Executar rollback controlado em simulação.
4. Realizar cutover final dos domínios definidos no escopo.
5. Encerrar com documentação completa + handover operacional.

Entregáveis semana 12

- Backend .NET ativo para fluxo essencial.
- RavenDB como store principal dos domínios migrados.
- Runbooks finais: deploy, rollback, incidentes e reconciliação.

5.1 Mapa operacional (Ondas x Semanas x Gate)

Para evitar leitura ambígua, este é o mapa oficial de ativação. Se o gate de uma onda falhar, a próxima não começa.

Onda	Semanas	Escopo principal	Gate obrigatório para avançar
1	1-2	Auth, sessão, infra base	auth estável + observabilidade mínima
2	3-4	Catálogo público e gestão de produtos	paridade catálogo + imagens válidas
3	5-6	Carrinho, cálculo e criação de pedido	comparador legado x .NET (3 execuções sem divergência em cenários críticos de totais/frete)
4	7-8	Pagamento, webhook e financeiro lojista	status financeiro consistente + reconciliação operacional
5	9-10	Painel admin, conteúdo e automações secundárias	painéis sem regressão crítica de autorização/dados
6	11-12	Canary, cutover e limpeza de legado	go/no-go final aprovado (Pedro + Operação + Financeiro) + rollback testado

6) Contrato de dados e migração MySQL → RavenDB

6.1 Estratégia de migração de dados

1. **Backfill inicial:** carga completa dos dados existentes para RavenDB.
2. **CDC incremental:** sincronização de alterações até o cutover.
3. **Validação de paridade:** amostragem + reconciliação por entidade.
4. **Freeze curto (se necessário):** janela breve para alinhar ponta final.

6.2 Definição de ownership de dados por fase

Fase	Fonte de escrita	Fonte de leitura principal	Observação
Fase 1 (fundação)	MySQL (legado)	MySQL	RavenDB preenchido por backfill.
Fase 2 (domínio migrado)	.NET + RavenDB	RavenDB	Legado consulta por compatibilidade temporária.
Fase 3 (cutover)	.NET + RavenDB	RavenDB	MySQL preservado para histórico e rollback curto.

6.3 Critérios de paridade de dados

- Pedidos: diferença zero em status e totais nos cenários críticos.
- Pagamentos: 100% de match em provider_id/status/montante.
- Produtos: 100% de match de disponibilidade e preço vigente.
- Usuários/lojas: 100% de match de relacionamento e permissões.

Meta objetiva: no go-live, a divergência aceitável fica restrita a campos não críticos de auditoria textual, nunca em valor financeiro ou status operacional.

6.4 Métrica de paridade (como medir sem achismo)

Domínio	Janela de medição	Critério de aprovação	Tolerância
Pedido (status/totais)	rolling 15 min + lote diário	match completo em cenários críticos	0 divergência financeira
Pagamento/split	evento a evento + fechamento diário	provider_id/status/montant e coerentes	0 divergência de valor
Antecipação/repassse	hora a hora + fechamento diário	saldo/limite/status iguais entre fontes	0 divergência crítica
Catálogo	amostragem por lote	disponibilidade/preço/ativo corretos	até 0,5% divergência não crítica por lote

7) Plano de APIs, versionamento e compatibilidade

7.1 Contrato de API

O frontend não pode quebrar por mudança de contrato no meio da migração. Portanto:

- Manter payloads compatíveis por versão.
- Adicionar campos novos como opcionais.
- Nunca remover campo sem depreciação anunciada.
- OpenAPI versionado e publicado a cada sprint.

7.2 Mapeamento de compatibilidade

Tipo	Regra	Validação
Resposta de pedidos	Paridade de totais, itens, frete e status por perfil	Contract tests + snapshots
Resposta financeira	Campos úteis por perfil e sem vazamento cruzado	Testes de autorização e payload
Erros de API	Mensagem clara e código HTTP consistente	Teste de casos negativos

7.3 Feature flags de roteamento

Vou usar flags por domínio para controlar migração sem deploy invasivo:

```
catalog.read = dotnet
catalog.write = dotnet
checkout.create_order = dotnet
payments.webhook = dotnet
orders.read_admin = dotnet
finance.anticipations = dotnet
```

8) Pagamentos, antecipações e reconciliação (área crítica)

Essa parte entra como crítica máxima. O desenho de migração precisa garantir:

- Idempotência de eventos de webhook.
- Persistência de referência externa (IDs da Pagar.me) para auditoria.
- Reconciliação periódica para cobrir falhas temporárias de webhook.
- Segregação por perfil para evitar vazamento de dados financeiros.

8.1 Regras de contrato financeiro durante a migração

Regra	Motivo	Risco se violar
Salvar payload mínimo + IDs externos	suporte e rastreabilidade	não conseguir auditar erro de pagamento
Não recalcular total no front	evitar divergência de cobrança	diferença de valor e contestação
Status financeiro por fonte oficial	consistência de painel	painel mentindo status de repasse
Fallback por reconciliação	resiliência operacional	pedido pago ficar como pendente

8.2 Pontos que ficam explicitamente pendentes de validação técnica

- Ajuste final de consistência eventual vs forte entre Payment e Order projections.
- Estratégia definitiva de particionamento de eventos financeiros em RavenDB.
- Padrão final para histórico financeiro de longo prazo e retenção.

Esses três pontos têm dono, prazo e gate no item **12.2**.

Importante: não vou travar o projeto esperando perfeição nesses três pontos. Vou avançar com baseline seguro e ajustar após a validação técnica, sem bloquear entregas das semanas 1-8.

9) Qualidade, testes e critérios de go/no-go

9.1 Pirâmide de testes obrigatória

- **Unitários:** regras de domínio e validações.
- **Integração:** persistência RavenDB, gateway de pagamento, filas/jobs.
- **Contrato:** compatibilidade de API com frontend atual.
- **Smoke E2E:** fluxos público/admin/lojista em staging.

9.2 Cenários de validação crítica

Fluxo	Critério de sucesso	Bloqueio de go-live
Checkout + pagamento PIX	pedido pago com status consistente em todos painéis	Sim
Pedido multi-loja	separação correta e visibilidade sem vazamento	Sim
Painel financeiro lojista	saldo/antecipação coerentes com fonte oficial	Sim
Admin pedidos	detalhamento completo com dados corretos	Sim
Catálogo público	listagem e detalhe sem regressão crítica	Não (desde que não afete checkout)

9.3 Go/No-Go do canary

1. Erro 5xx abaixo do baseline legado.
2. Sem diferença de totais em pedidos de teste.
3. Sem perda de eventos de webhook.
4. Sem regressão crítica de autorização por perfil.
5. Rollback testado com sucesso no mesmo ambiente.

9.4 Gates intermediários (obrigatórios antes do canary final)

Semana	Gate	Critério objetivo	Se falhar
Semana 6	Gate Onda 3	checkout cria pedido com totais/frete corretos em 100% da suíte crítica	bloquear avanço para pagamento/webhook
Semana 8	Gate Onda 4	pagamento + webhook + reconciliação sem divergência financeira crítica	bloquear avanço para painéis e canary
Semana 10	Gate Onda 5	painel admin/lojista/cliente sem regressão crítica de autorização e dados	bloquear entrada na janela final de cutover

10) Observabilidade, segurança e operação

10.1 Observabilidade

- Logs estruturados por correlation_id e request_id.
- Métricas: p50/p95, throughput, taxa de erro por endpoint, filas pendentes.
- Eventos de negócio: pedido criado, pedido pago, reconciliação executada.
- Alertas: aumento de erro, queda de throughput, divergência financeira.

10.2 Segurança

- Segredos fora de código (env/secret manager).
- Princípio de menor privilégio para banco e integrações.
- Auditoria de ações administrativas sensíveis.
- Proteção de dados pessoais conforme LGPD.

10.3 Operação

- Playbook de incidente por severidade (S1, S2, S3).
- Rotina de backup com teste de restauração semanal.
- Healthchecks ativos em staging e produção.
- Deploy com validação automática e rollback de artefato.

10.4 Backup e restauração (escopo, dono e evidência)

Componente	Frequência	Retenção	Dono	Teste de restore	Evidência
MySQL (legado)	diário	30 dias	Pedro	semanal em staging isolado	log + checksum + timestamp
RavenDB (novo backend)	diário	30 dias	Pedro	semanal em staging isolado	restore validado + relatório
Arquivos críticos (mídia/config)	diário	30 dias	Pedro	quinzenal por amostra	relatório de restauração

10.5 Scheduler e severidade operacional (S1/S2/S3)

- **Scheduler obrigatório:** orders:reconcile, financial:reconcile-balances, ops:monitor-health.
- **SLO de execução:** nenhuma rotina crítica pode ficar sem execução por mais de 10 minutos.
- **Alerta de job perdido:** ao detectar missed run, abrir incidente automaticamente e acionar on-call.

Severidade	Definição	Resposta inicial	Dono
S1	risco direto de receita, pagamento, vazamento de dados ou indisponibilidade crítica	ação imediata + rollback se necessário	Pedro
S2	degradação relevante sem perda financeira imediata	mitigação em até 30 min	Pedro
S3	incidente de baixo impacto sem bloqueio operacional	correção planejada no ciclo	Pedro

11) Cutover, rollback e continuidade

11.1 Estratégia de cutover

- 1. Canary por percentual de tráfego ou por rota.
- 2. Subida gradual de escopo: catálogo → checkout → pagamentos → financeiro.
- 3. Janela de observação em cada etapa antes de ampliar.

11.2 Estratégia de rollback

- Rollback por feature flag (primeira linha).
- Rollback de deploy de API (segunda linha).
- Plano de contingência manual para pagamento/pedido (terceira linha).

11.3 Matriz de decisão rápida

Sintoma	Ação em até 5 min	Ação em até 30 min
Aumento de erro em checkout	voltar flag checkout para legado	analisar logs e reenfileirar fluxos interrompidos
Status de pagamento inconsistente	ativar reconciliação imediata	validar idempotência e eventos duplicados
Dados divergentes em painel	forçar leitura da fonte oficial	corrigir projection e reprocessar eventos

11.4 Runbook de rollback de dados (RavenDB + financeiro)

- 1. Congelar escrita no domínio afetado e forçar modo somente leitura.
- 2. Desativar flags do domínio e retornar leitura/escrita para legado.
- 3. Congelar CDC incremental e registrar cursor de interrupção.
- 4. Executar rollback técnico de coleções impactadas no RavenDB (ou reset para último snapshot validado).
- 5. Rodar reconciliação de pedidos e financeiro para zerar pendências do intervalo.
- 6. Publicar relatório curto: janela afetada, impacto, dados reprocessados, status final.

Regra prática: rollback de funcionalidade sem rollback de dados não é considerado rollback completo.

12) Riscos, mitigação e decisões pendentes

12.1 Registro de riscos

Risco	Prob.	Impacto	Mitigação
Escopo expandir durante as 12 semanas	Média	Alto	Change control semanal e freeze de ciclo
Divergência MySQL x RavenDB	Média	Alto	Backfill + CDC + reconciliação diária
Falha em webhook de pagamento	Alta	Alto	Idempotência + fallback de reconciliação
Cutover em janela ruim	Baixa	Alto	Canary progressivo + rollback ensaiado

12.2 Decisões pendentes (com dono, prazo e gate)

Decisão	Dono	Prazo	Gate impactado	Bloqueia cutover?
Modelo final de documentos financeiros no RavenDB	Pedro	Semana 8	Gate Onda 4	Sim
Nível final de consistência para projections críticos	Pedro	Semana 8	Gate Onda 4	Sim
Particionamento de eventos financeiros no RavenDB	Pedro	Semana 8	Gate Onda 4	Sim
Retenção de eventos de auditoria	Pedro	Semana 10	Gate Onda 5	Não (desde que exista retenção mínima temporária)

Esses itens entram em refinamento técnico final. O plano já prevê avanço sólido sem travar cronograma por isso.

13) Orçamento de esforço e distribuição semanal

13.1 Distribuição macro (estimativa realista)

Bloco	% esforço	Semanas foco
Fundação e arquitetura base	15%	1-2
Catálogo e dados mestres	20%	3-4
Checkout e pedidos	25%	5-6
Pagamento e financeiro	25%	7-8
Hardening, canary e cutover	15%	9-12

13.2 Cadência de gestão

- Planejamento semanal com backlog fechado.
- Daily curta de bloqueios técnicos.
- Review semanal com evidência de entrega em staging.
- Checkpoint quinzenal de arquitetura.

14) Anexos operacionais

Anexo A - Checklist semanal mínimo

- Build e deploy em staging sem erro.
- Smoke test público/admin/lojista executado.
- Endpoints novos com contract test.
- Métricas da semana comparadas com baseline.
- Riscos atualizados e plano de mitigação revisado.

Anexo B - Checklist de mudança de domínio para .NET

1. Contrato OpenAPI atualizado.
2. Paridade funcional validada por cenário crítico.
3. Logs e métricas do domínio observáveis.
4. Feature flag pronta para rollback imediato.
5. Playbook de incidente específico desse domínio publicado.

Anexo C - Checklist de cutover final

1. Backfill final + sincronização incremental sem erro.
2. Canary estável por período mínimo definido.
3. Sem divergência de totais de pedido e status de pagamento.
4. On-call técnico alinhado com plano de rollback.
5. Comunicação interna pronta para janela de transição.

Anexo D - Métricas de sucesso pós-cutover (30 dias)

Métrica	Meta	Observação
Disponibilidade API	>= 99,9%	considerando janelas planejadas
Erro 5xx	igual ou menor que legado	monitoramento diário
Paridade de status de pedido	100% cenários críticos	amostragem e reconciliação
Divergência financeira	0 em valor crítico	qualquer divergência abre incidente

15) WBS detalhado (Work Breakdown Structure)

O objetivo aqui é quebrar o plano em pacotes executáveis, com dono e saída objetiva. Isso reduz ruído de gestão e evita tarefa “aberta demais”.

ID	Pacote de trabalho	Saída esperada	Dependência
W1	Inventário técnico legado	Mapa de endpoints/jobs/webhooks	-
W2	Scaffold .NET e CI/CD	API buildando e deployando em staging	W1
W3	Setup RavenDB	cluster staging, índices iniciais, backups	W2
W4	Contrato OpenAPI inicial	spec versionada para domínios críticos	W1
W5	Backfill de catálogo	lojas/produtos no RavenDB	W3
W6	Catálogo read no .NET	listagens estáveis com paridade	W4, W5
W7	Catálogo write no .NET	cadastro/edição sem regressão	W6
W8	Carrinho/cálculo no .NET	totais/frete/validação paritários	W6
W9	Criação de pedido	checkout criando pedido no .NET	W8
W10	Pagamento e webhook	status de pagamento consistente	W9
W11	Reconciliação	fallback automático operacional	W10
W12	Financeiro lojista/admin	painéis com dados confiáveis	W10, W11
W13	Canary progressivo	go-live parcial sob monitoramento	W12
W14	Cutover final	domínios principais no novo backend	W13

Nota de amarração: W8 e W9 pertencem à mesma janela de execução (Semanas 5-6), em sequência obrigatória: primeiro cálculo/carrinho (W8), depois criação de pedido (W9).

15.1 Definição de pronto por pacote

- Commit validado em staging, com build íntegro.
- Smoke test de fluxo afetado + adjacente.
- Dashboard de métrica atualizado para o pacote.
- Rollback documentado do pacote.

16) Modelo operacional de equipe e responsabilidades

Mesmo com execução central minha, é essencial distribuir responsabilidade por “frente” para não virar gargalo de decisão.

Frente	Responsável primário	Responsável de apoio	Indicador semanal
Arquitetura/API	Pedro	Validação técnica	número de endpoints com contrato estável
Modelagem RavenDB	Pedro	Validação técnica	latência de leitura + custo de escrita
Pagamentos/financeiro	Pedro	Operação admin	conciliação sem divergência crítica
Qualidade e regressão	Pedro	QA assistido	taxa de falha por suíte smoke
Operação/infra	Pedro	DevOps de suporte (quando necessário)	disponibilidade + MTTR

16.1 Rituais de controle (sem burocracia inútil)

1. Daily de 15 minutos focada em impedimentos reais.
2. Review semanal com demo obrigatória do que foi migrado.
3. Checkpoint quinzenal de decisões estruturais.
4. Atualização rápida para o negócio (status, risco, próximo passo).

17) Mapeamento de entidades e estratégia de migração por dado

Para reduzir risco de perda de contexto, a migração não é só técnica. É semântica de negócio. Abaixo, como cada entidade será tratada.

Entidade atual (MySQL)	Destino (RavenDB)	Tipo de migração	Validação
users	Users/{id}	backfill + sync incremental	amostra + hash de campos críticos
stores	Stores/{id}	backfill + owner no novo backend	paridade de vínculos com user
products	Products/{id}	backfill em lotes	preço/estoque/ativo por loja
orders	Orders/{id}	misto (histórico + novos no .NET)	totais/status por pedido
order_items	embed em Orders	transformação orientada a leitura	contagem de itens e valores
withdraw / financial	FinancialMovements/{id}	migração progressiva	saldo disponível e histórico

17.1 Política de integridade

- Nenhum pedido “novo” será escrito em duas fontes sem idempotência e auditoria.
- Campos financeiros sensíveis terão validação dupla por rotina diária.
- Divergência em valor monetário abre incidente S1 automaticamente.

17.2 Plano de reconciliação de dados

1. Reconciliação diária automatizada por entidade crítica.
2. Relatório de divergência com severidade e prioridade de correção.
3. Rotina de auto-correção para inconsistências simples (campos de projeção).
4. Escalonamento manual para inconsistências de valor financeiro.

18) Backlog tático de 90 dias (sprint a sprint)

O backlog detalhado foi consolidado nos itens **5** (execução por semana) e **15** (WBS com dependências) para evitar duplicidade e reduzir ruído de manutenção.

Regra prática: toda atualização de cronograma acontece apenas nesses dois itens.

19) Plano de cutover hora a hora (janela final)

Modelo de execução para não improvisar no dia crítico:

Hora	Ação	Dono	Critério de validação
T-3h	Congelar mudanças não essenciais	Pedro	Sem deploy paralelo em andamento
T-2h	Rodar reconciliação final de dados	Pedro	Divergência crítica = 0
T-90m	Rodar suíte smoke completa	Pedro	Sem falha crítica
T-60m	Habilitar canary inicial	Pedro	erro 5xx ≤ baseline legado (referência: média dos últimos 14 dias)
T-30m	Expandir canary para rotas prioritárias	Pedro	Sem regressão de checkout
T	Cutover principal	Pedro	Métricas estáveis 30 min
T+30m	Auditoria de pedidos/pagamentos	Pedro	Paridade total nos cenários críticos
T+60m	Formalizar encerramento da janela	Pedro	Sem incidente aberto S1/S2

19.1 Critérios de rollback imediato (sem discussão longa)

- Erro 5xx acima de 2x baseline por 5 minutos.
- Falha de criação de pedido acima de limiar definido.
- Divergência financeira em pedido novo.
- Falha de autorização com risco de exposição de dados.

19.2 Aprovação formal de go/no-go (signoff)

- **Comitê mínimo:** Pedro (Tech Lead), responsável de Operação e responsável de Financeiro.
- **Artefatos obrigatórios:** relatório de métricas do canary, checklist de rollback validado, relatório de risco residual.
- **Regra:** sem os três artefatos, o cutover não é aprovado.

Regra operacional: se bater critério de rollback, rollback executa primeiro e análise pós-incidente vem depois. Não se debate com produção degradando.

20) Plano de comunicação e governança da mudança

20.1 Comunicação interna

- Canal único de execução da migração (sem mensagens soltas em múltiplos lugares).
- Atualização diária curta com: status, risco, bloqueio e decisão necessária.
- Registro de incidentes e decisões no mesmo repositório de documentação.

20.2 Comunicação com negócio

- Resumo semanal: o que entrou, o que saiu, o que ainda preocupa.
- Status de risco traduzido para impacto de receita/operação.
- Sem jargão técnico em atualização para direção.

21) Checklist de validação pontual

Aqui eu deixo só 2 checkpoints externos: 1 para RavenDB e 1 para .NET. É o suficiente para validar os pontos críticos sem perder ritmo de entrega.

Semana	Tópico da revisão	Responsável	Saída esperada
Semana 4	Checkpoint 1: arquitetura .NET, boundaries e consistência de checkout/pedido/pagamento	Validação técnica	ajuste estrutural do fluxo transacional e anti-patterns
Semana 8	Checkpoint 2: modelagem RavenDB para catálogo + financeiro/eventos críticos	Validação técnica	direção final de índices, shape de documentos e leitura financeira

21.1 O que já posso decidir sem bloquear por validação técnica

- Contrato de API e compatibilidade com frontend.
- Pipeline de qualidade e critérios de aceite por sprint.
- Estratégia de canary/rollback e governança de mudança.
- Sequência de migração dos domínios por risco de negócio.

22) Plano de hardening pós-go-live (primeiros 30 dias)

O projeto não acaba no cutover. Aqui é onde consolido estabilidade e removo dívida gerada durante a corrida de migração.

22.1 Semana +1 pós-go-live

- Monitoramento intensivo de erros e latência por endpoint.
- Correção rápida de regressões não críticas de UX/detalhe.
- Revisão diária de reconciliação financeira.

22.2 Semana +2 pós-go-live

- Otimização de índices e consultas de maior custo.
- Limpeza de flags temporárias que não fazem mais sentido.
- Consolidação de documentação final de operação.

22.3 Semana +3 e +4 pós-go-live

- Desativação planejada do legado para domínios migrados.
- Checklist de segurança e compliance de dados atualizado.
- Fechamento de dívida técnica de migração (top 10 itens).

Saída esperada em 30 dias: plataforma estável no novo backend, operação confortável, sem dependência de “gambiarra de transição”.

23) Inventário funcional da Fiestou (feature por feature)

Esta é a lista contratual macro das funcionalidades que entram no remake. A ideia é evitar “furo de escopo escondido”. Cada item tem onda de migração e criticidade.

Fonte oficial de escopo: esta seção (23) é a referência principal de aprovação.

23.1 Público (site aberto / funil de aquisição)

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
PUB-01	Home com seções dinâmicas	/	content/home	Média	2
PUB-02	Listagem geral de produtos	/produtos	request/products	Alta	2
PUB-03	Paginação de produtos	/produtos/pagina/[page]	request/products	Média	2
PUB-04	Listagem com filtros	/produtos/listagem	request/products	Alta	2
PUB-05	Detalhe de produto	/produtos/[id]	request/product	Crítica	2
PUB-06	Seleção de variações/adicionais	/produtos/[id]	products/register (estrutura)	Crítica	3
PUB-07	Cálculo de frete por CEP	/produtos/[id]	delivery-zipcode/*	Crítica	3
PUB-08	Regras de locação por loja	/produtos/[id]	store.rental_rules	Alta	3
PUB-09	Produtos relacionados (“veja também”)	related-products	request/products	Média	4
PUB-10	Rastreamento de interesse de produto	request/product-interest	ProductInterestController	Média	4
PUB-11	Recomendação de produto	request/product-recommendations	ProductInterestController	Média	4
PUB-12	Páginas de categorias	/categoria/[slug]	request/category	Média	2
PUB-13	Perfil público de parceiro/loja	/[slug]	request/store	Alta	2
PUB-14	Blog	/blog	content/blog	Baixa	5
PUB-15	Post de blog	/blog/[slug]	content/post	Baixa	5

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
PUB-16	FAQ	/faq	content/faq	Baixa	5
PUB-17	Sobre	/sobre	content/about	Baixa	5
PUB-18	Contato	/contato	content/contact	Baixa	5
PUB-19	Comunicados/termos	/comunicados/[slug]	content/communicate	Média	5
PUB-20	Cadastro de parceiro (landing)	/parceiros/suja-parceiro	auth/pre-register	Média	4

23.2 Conta, autenticação e jornada de cliente

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
ACC-01	Login padrão	/acesso	auth/login	Crítica	1
ACC-02	Login restrito (admin)	/acesso/restrito	auth/login	Crítica	1
ACC-03	Login entregador	/acesso/entregador	auth/login	Média	4
ACC-04	Cadastro cliente	/cadastre-se	auth/register	Alta	1
ACC-05	Completar cadastro	/cadastre-se/completar	users/update	Alta	1
ACC-06	Recuperar senha	/recuperar	auth/recovery	Média	4
ACC-07	Redefinir senha	/recuperar/senha	auth/redefine	Média	4
ACC-08	Logout	/logout	logout	Alta	1
ACC-09	Sessão autenticada	middleware	auth/me + refresh	Crítica	1
ACC-10	Painel cliente - visão geral	/dashboard	orders/list, users/get	Alta	4
ACC-11	Painel cliente - pedidos	/dashboard/pedidos	orders/list	Alta	4
ACC-12	Painel cliente - detalhe pedido	/dashboard/pedidos/[id]	order/{group hash}	Alta	4
ACC-13	Painel cliente - meus dados	/dashboard/meus-dados	users/get/update	Média	5
ACC-14	Painel cliente - endereços	/dashboard/enderecos	users/get/update	Média	5
ACC-15	Painel cliente - favoritos	/dashboard/favoritos	account/user	Média	5

23.3 Fluxo comercial (carrinho, checkout, pedido)

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
COM-01	Carrinho persistido por sessão	/carrinho	cart GET/POST/DELETE	Crítica	3
COM-02	Resumo por loja no carrinho	/carrinho	cálculo server-side	Crítica	3
COM-03	Validação de mínimo por loja	/carrinho /checkout	store.minimum_order_value	Alta	3
COM-04	Checkout multi-loja	/checkout	checkout/create	Crítica	3
COM-05	Cadastro/edição de dados no checkout	/checkout	users/update	Alta	3
COM-06	Criação de pedido e itens	checkout finalização	orders/register	Crítica	3
COM-07	Metadados do pedido	checkout finalização	orders/register-meta	Alta	3
COM-08	Confirmação pagamento	checkout/pay return	orders/processing	Crítica	4
COM-09	Webhook Pagar.me	/hooks/pagar.me	HooksController	Crítica	4
COM-10	Reconciliação de pedidos	scheduler	orders:reconcile	Crítica	4
COM-11	Status de entrega do pedido	painel pedidos	orders/update-delivery-status	Alta	4
COM-12	Comentários/avaliação de pedido	dashboard/pedidos/[id]	comments/register	Média	5

23.4 Painel do lojista (vendas e operação)

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
PAR-01	Dashboard lojista	/painel	stores/balance, stores/products	Alta	4
PAR-02	Cadastro/edição da loja	/painel/loja	stores/form/register	Alta	4
PAR-03	Configuração de entrega por CEP/cidade	/painel/loja	zipcode-cities-range*	Média	5
PAR-04	Listagem de produtos da loja	/painel/produtos	stores/products	Alta	2
PAR-05	Cadastro/edição de produto	/painel/produtos/[id]	products/register	Crítica	2
PAR-06	Upload/reordenação de galeria	/painel/produtos/[id]	products/upload-gallery/reorder	Alta	2
PAR-07	Importação de produtos	/painel/produtos/importar	products/import-preview/import	Média	5
PAR-08	Listagem de pedidos da loja	/painel/pedidos	orders/list	Crítica	4
PAR-09	Detalhe de pedido da loja	/painel/pedidos/[id]	orders/get	Crítica	4
PAR-10	Status de entrega pelo lojista	/painel/pedidos/[id]	orders/update-delivery-status	Alta	4
PAR-11	Gestão de clientes da loja	/painel/clientes	stores/customers	Média	5
PAR-12	Dados do recebedor	/painel/dados_do_recebedor	recipient/* + info/recipients/status	Crítica	4
PAR-13	Painel financeiro (saldos)	/painel/financeiro	info/financial/overview	Crítica	4

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
PAR-14	Antecipações do lojista	/painel/financeiro	info/financial/anticipations*	Crítica	4
PAR-15	Configuração antecipação automática	/painel/financeiro	info/financial/automatic-anticipation-settings	Alta	4
PAR-16	Configuração de repasse	/painel/financeiro	info/transfer-settings	Alta	4

23.5 Painel admin (operação, conteúdo e risco)

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
ADM-01	Dashboard admin	/admin	admin/dashboard/stats	Alta	5
ADM-02	Refresh status operacional	/admin	admin/dashboard/refresh-operational-status	Alta	5
ADM-03	Listagem de pedidos	/admin/pedidos	orders/list	Crítica	4
ADM-04	Detalhe de pedido completo	/admin/pedidos/[id]	orders/get + request/product	Crítica	4
ADM-05	Parceiros (listagem)	/admin/parceiros	users/list	Média	5
ADM-06	Parceiro (detalhe/pagamento)	/admin/parceiros/[id]	admin/partner/payment	Média	5
ADM-07	Usuários	/admin/usuarios	users/list/get	Média	5
ADM-08	Entregadores	/admin/entregadores	users/list	Média	5
ADM-09	Carrinhos abandonados	/admin/carrinhos-abandonados	admin/abandoned-carts*	Alta	5
ADM-10	Recomendações e interesses	/admin/recomendacoes	admin/recommendations*	Média	5
ADM-11	Antecipações de lojistas	/admin/antecipacoes	admin/anticipations*	Alta	5
ADM-12	Gestão de conteúdo geral	/admin/contudo	admin/content*	Média	5
ADM-13	Gestão de comunicados	/admin/comunicados	admin/content*	Média	5
ADM-14	Gestão de blog	/admin/blog	admin/content*	Média	5
ADM-15	Regras de negócio	/admin/sistema	admin/content/get/regist	Média	5

ID	Feature	Página/rota atual	Backend atual	Criticidade	Onda
	(sistema)		er		
ADM-16	Filtro por grupos/elementos	/admin/filtro	group/* + element/*	Média	5
ADM-17	Configuração de entrega	/admin/configuracoes-entrega	zipcode-cities-range*	Média	5
ADM-18	Configurações de sistema	/admin/configuracoes	admin/settings/*	Alta	5
ADM-19	Templates transacionais	/admin/templates	admin/templates/*	Média	5

23.6 Automação, jobs, telemetria e manutenção

ID	Feature	Componente atual	Criticidade	Onda	Observação de migração
OPS-01	Notificação de carrinho abandonado	app:notify-abandoned-carts	Média	5	depende SMTP estável
OPS-02	Reconciliação de pedidos	orders:reconcile	Crítica	4	core de fallback pagamento
OPS-03	Processamento de créditos de loja	credits:process	Alta	4	financeiro sensível
OPS-04	Reconciliação financeira diária	financial:reconcile-balances	Alta	4	base para admin dashboard
OPS-05	Monitor de saúde operacional	ops:monitor-health	Alta	5	alertas automáticos
OPS-06	Limpeza de telemetria	maintenance:cleanup-telemetry	Média	5	controle de crescimento do banco
OPS-07	Webhook Pagar.me	hooks/pagarme	Crítica	4	idempotência obrigatória
OPS-08	Webhook Stripe (legado)	hooks/stripe	Baixa	6	validar permanência real
OPS-09	Healthcheck front prod	fiestou-prod-healthcheck	Alta	1	já ativo em produção
OPS-10	Healthcheck staging dev	fiestou-staging-healthcheck	Alta	1	evita quebra de chunks

23.7 Consolidação de ondas de migração

- **Onda 1:** auth, sessão, infra base, observabilidade.
- **Onda 2:** catálogo público e gestão de produtos.
- **Onda 3:** carrinho, cálculo e checkout.
- **Onda 4:** pedidos, pagamento, webhook, financeiro lojista.
- **Onda 5:** admin completo, conteúdo, automações secundárias.
- **Onda 6:** limpeza de legado e pós-cutover.

Para execução e aprovação, o cronograma oficial dessas ondas está no item **5.1** (Ondas x Semanas x Gate).

24) Ordem de importação de dados (MySQL → RavenDB) detalhada

Esta ordem evita quebra de referência durante backfill e permite validar domínio por domínio. Aqui está o plano com sequência explícita.

Ordem	Tabela atual	Documento/col eção destino	Dependência	Modo	Critério de validação
1	user	Users	-	full backfill	100% ids e tipos
2	store	Stores	user	full backfill	store.user válido
3	recipients	Recipients	store	full backfill	recipient_code íntegro
4	recipient_configs	Recipients.config	recipients	merge	flags financeiras corretas
5	recipient_addresses	Recipients.addresses	recipients	merge	endereço completo
6	recipient_phones	Recipients.phones	recipients	merge	DDD+número válidos
7	recipient_partners	Recipients.partners	recipients	merge	documentos sem duplicidade
7A	recipient_configs.transfer_settings (ou estrutura equivalente)	TransferSettings	recipients/store	extract + backfill	política de repassse íntegra por loja
8	group	Groups	-	full backfill	hierarquia consistente
9	elements	Elements	group	full backfill	vínculo de grupo válido
10	group_elements/ elements_rel	GroupMappings	group/elements	transform	contagem compatível
11	product	Products	store	full backfill em lote	preço/comercialType/status
12	media	MediaAssets	product/store	full backfill	url acessível
13	media_rel	Products.media	media/product	merge	ordem de galeria íntegra
14	category/category_rel (legado)	LegacyCategory Map	product	optional	somente referência histórica
15	zipcode_cities_ranges	ZipRules	-	full backfill	faixas válidas
16	zipcode_cities_range_stores	StoreZipRules	store/ziprules	merge	mapeamento por loja

Ordem	Tabela atual	Documento/col eção destino	Dependência	Modo	Critério de validação
17	cart_sessions	Carts	user/product	incremental	sessões recentes apenas
18	order (headers)	Orders	user/store	backfill em janela	id/group_hash/st atus
19	order_items	Orders.items	order/product	embed	itens e quantidades
20	order_item_addo ns	Orders.items.add ons	order_items	embed	valor de adicional íntegro
21	freights_orders_ price	Orders.shipping Price	order	merge	frete por pedido
22	comment	OrderComments	order/user	full backfill	referência de pedido
23	withdraw	FinancialMovem ents	store/recipient	transform	tipo e status normalizados
24	store_credits	StoreCredits	store/order	full backfill	pending/transferr ed corretos
24A	antecipações (snapshot financeiro + referência externa)	FinancialAnticipa tions	recipient/store	backfill técnico	status e limite por recebedor coerentes
25	product_interest	ProductInterests	product/user	full backfill	actor e evento íntegro
26	product_interest _actor_ip	InterestActors	product_interest	merge	IP hash e vínculo válidos
27	product_search_i nterest	SearchInterests	user/session	full backfill	query normalizada
28	content/content_ rel	CmsContent	-	full backfill	slug e type únicos
29	message_templa tes	MessageTemplat es	-	full backfill	slug ativo
30	system_settings	SystemSettings	-	full backfill	segredos preservados

24.1 Importação incremental (CDC) após backfill

1. Aplicar CDC contínuo para order, order_items, cart_sessions, withdraw, transfer_settings e financial_anticipations.
2. Aplicar CDC de baixa frequência para conteúdo e dados estáticos.
3. Validar checkpoint por lote com marcação de cursor de migração.
4. Congelar escrita dual antes do cutover final.

24.2 Ordem de ativação por domínio (runtime)

Fase runtime	Fonte de escrita	Fonte de leitura	Plano de retorno
Fase A	Laravel/MySQL	Laravel/MySQL	-
Fase B (catálogo)	.NET/RavenDB	.NET/RavenDB	flag catálogo para legado
Fase C (checkout/pedido)	.NET/RavenDB	.NET/RavenDB	flag checkout para legado
Fase D (pagamentos)	.NET/RavenDB	.NET/RavenDB	rollback webhook + reconciliação
Fase E (admin completo)	.NET/RavenDB	.NET/RavenDB	retorno por módulo admin

25) Incidentes e prevenção (resumo objetivo)

Esta seção foi reduzida para evitar duplicidade com o item 28. O foco aqui é só o que precisa ser coberto como regra de engenharia.

1. Integrações financeiras externas devem operar com resiliência: retry controlado, cache curto e modo degradado seguro.
2. Fluxos de pedido/pagamento exigem idempotência e reconciliação automática para evitar inconsistência de status.
3. Deploy precisa ser atômico com healthcheck pós-release para prevenir quebra de chunks/manifests.
4. Migração de schema só avança com validação automática de compatibilidade no startup.
5. Painéis críticos precisam de consultas estáveis e monitor de latência para manter operação fluida.

26) Cobertura funcional completa (1 por 1) para o remake

Esta seção congela o escopo funcional real da Fiestou no momento atual. A meta é simples: tudo que está aqui precisa ter equivalente no remake, com contrato compatível e teste de aceite.

Leitura correta: esta seção é o detalhamento operacional derivado da seção 23 (não é um segundo escopo concorrente).

26.1 Mapa de páginas públicas e aquisição

ID	Rota/página	Objetivo de negócio	Dependência principal	Onda
PG-PUB-01	/	Landing e aquisição	content/home + products	2
PG-PUB-02	/produtos	Catálogo principal	request/products	2
PG-PUB-03	/produtos/pagina/[page]	Paginação de catálogo	request/products	2
PG-PUB-04	/produtos/listagem	Busca e filtros	request/products + groups	2
PG-PUB-05	/produtos/[id]	Conversão de produto	request/product + delivery	3
PG-PUB-06	/categoria/[slug]	Navegação temática	request/products + segment	2
PG-PUB-07	/categoria/[slug]/pagina/[page]	Paginação de categoria	request/products	2
PG-PUB-08	/[slug]	Loja pública	request/store + request/products	2
PG-PUB-09	/carrinho	Pré-checkout	cart + delivery rules	3
PG-PUB-10	/checkout	Conversão em pedido	checkout/create + order/register	3
PG-PUB-11	/finalizar	Pós-pagamento	status de pagamento	4
PG-PUB-12	/blog	Aquisição orgânica	content/blog	5
PG-PUB-13	/blog/[slug]	Conteúdo e SEO	content/post	5
PG-PUB-14	/faq	Suporte pré-venda	content/faq	5
PG-PUB-15	/sobre	Confiança de marca	content/about	5
PG-PUB-16	/contato	Canal comercial	content/contact	5
PG-PUB-17	/comunicados/[slug]	Comunicação oficial	content/communicate	5
PG-PUB-18	/parceiros	Aquisição de lojistas	content/partners	5
PG-PUB-19	/parceiros/seja-parceiro	Lead de lojista	content/become-partner	4

ID	Rota/página	Objetivo de negócio	Dependência principal	Onda
PG-PUB-20	/parceiros/cadastro	Onboarding lojista	auth/pre-register + stores/completeregister	4
PG-PUB-21	/cadastre-se	Cadastro de cliente	auth/register	1
PG-PUB-22	/cadastre-se/completar	Completar onboarding	auth/pre-register + users/update	1
PG-PUB-23	/acesso	Login geral	auth/login	1
PG-PUB-24	/auth	Callback de autenticação	auth/external-auth	1
PG-PUB-25	/recuperar	Recuperação de conta	auth/recovery	4
PG-PUB-26	/recuperar/senha	Reset de senha	auth/redefine	4
PG-PUB-27	/logout	Encerrar sessão	auth/logout	1
PG-PUB-28	/entregador	Entrada de entregadores	content/roles	5
PG-PUB-29	/entregador/seja-entregador	Aquisição logística	content/register	5
PG-PUB-30	/entregador/cadastro	Cadastro entregador	users/register/update	5

26.2 Mapa de páginas do cliente autenticado

ID	Rota/página	Objetivo	Dependência principal	Onda
PG-CLI-01	/dashboard	Resumo de conta	orders/list + users/get	4
PG-CLI-02	/dashboard/pedidos	Histórico de pedidos	orders/list	4
PG-CLI-03	/dashboard/pedidos/[id]	Detalhe do pedido	order/{grouphash}	4
PG-CLI-04	/dashboard/pedidos/pagamento/[id]	Status de pagamento	orders/check-payment	4
PG-CLI-05	/dashboard/conta	Conta base	users/get/update	5
PG-CLI-06	/dashboard/meus-dados	Dados pessoais	users/get/update	5
PG-CLI-07	/dashboard/endereços	Endereços	users/update	5
PG-CLI-08	/dashboard/favoritos	Interesse/favoritos	request/account/user	5
PG-CLI-09	/dashboard/chat	Canal de atendimento	chat/infra	6

26.3 Mapa de páginas do painel lojista

ID	Rota/página	Objetivo	Dependência principal	Onda
PG-PAR-01	/painel	Visão operacional	orders/dashboard-stats + stores/balance	4
PG-PAR-02	/painel/conta	Conta do lojista	users/get/update	5
PG-PAR-03	/painel/loja	Configurar loja	stores/form/register	4
PG-PAR-04	/painel/produtos	Portfólio de produtos	stores/products	2
PG-PAR-05	/painel/produtos/[id]	Criar/editar produto	products/register + gallery	2
PG-PAR-06	/painel/produtos/importar	Importação em lote	products/import-preview/import	5
PG-PAR-07	/painel/pedidos	Lista de pedidos	orders/list	4
PG-PAR-08	/painel/pedidos/[id]	Gestão de pedido	orders/get + update-delivery-status	4
PG-PAR-09	/painel/clientes	Base de clientes	stores/customers	5
PG-PAR-10	/painel/clientes/[id]	Detalhe cliente	stores/customers	5
PG-PAR-11	/painel/financeiro	Saldo e antecipações	info/financial/*	4
PG-PAR-12	/painel/dados_dono_recebedor	Onboarding de split	recipient/* + transfer-settings	4
PG-PAR-13	/painel/chat	Canal direto	chat/infra	6
PG-PAR-14	/painel/saques (legado)	Rota de compatibilidade	redirect/legacy	6

26.4 Mapa de páginas do painel admin

ID	Rota/página	Objetivo	Dependência principal	Onda
PG-ADM-01	/admin	Dashboard admin	admin/dashboard/status	5
PG-ADM-02	/admin/pedidos	Operação de pedidos	orders/list	4
PG-ADM-03	/admin/pedidos/[id]	Detalhe técnico/comercial	orders/get	4
PG-ADM-04	/admin/parceiros	Gestão de lojistas	users/list	5
PG-ADM-05	/admin/parceiros/[id]	Dados financeiros parceiro	admin/partner/payment	5
PG-ADM-06	/admin/usuarios	Gestão de usuários	users/list	5
PG-ADM-07	/admin/usuarios/[id]	Detalhe de usuário	users/get	5
PG-ADM-08	/admin/entregadores	Operação logística	users/list (type=entregador)	5
PG-ADM-09	/admin/antecipacoes	Controle financeiro admin	admin/anticipations*	5
PG-ADM-10	/admin/recomendacoes	Inteligência comercial	admin/recommendations*	5
PG-ADM-11	/admin/carrinhos-abandonados	Recuperação de conversão	admin/abandoned-carts*	5
PG-ADM-12	/admin/conteudo	CMS central	admin/content*	5
PG-ADM-13	/admin/conteudo/[slug]	Edição de conteúdo	admin/content*	5
PG-ADM-14	/admin/comunicados	Comunicados institucionais	admin/content*	5
PG-ADM-15	/admin/comunicados/[slug]	Edição de comunicado	admin/content*	5
PG-ADM-16	/admin/blog	Blog institucional	admin/content*	5
PG-ADM-17	/admin/blog/[id]	Edição de post	admin/content*	5

ID	Rota/página	Objetivo	Dependência principal	Onda
PG-ADM-18	/admin/configuracoes	Configuração sistêmica	admin/settings/*	5
PG-ADM-19	/admin/configuracoes/gmail-callback	OAuth Gmail	admin/settings/gmail/*	5
PG-ADM-20	/admin/configuracoes-entrega	Regras de entrega	zipcode-cities-range*	5
PG-ADM-21	/admin/templates	Templates transacionais	admin/templates/*	5
PG-ADM-22	/admin/sistema	Regras gerais de negócio	admin/content/get/register	5
PG-ADM-23	/admin/filtro	Taxonomia de filtros	group/* + element/*	5
PG-ADM-24	/admin/saques e /admin/saques/[slug] (legado)	Compatibilidade de rota	redirect/legacy	6

26.5 Cobertura de API por domínio (inventário de contrato)

Domínio	Endpoints atuais (resumo 1:1)	Prioridade no remake	Risco de quebra
Autenticação	auth/checkin, auth/login, auth/external-auth, auth/register, auth/pre-register, auth/recovery, auth/redefine, refresh, logout, me	Onda 1	Crítico
Usuários	users/get, users/update, users/list, users/validate, relacionamento familiar	Onda 1/5	Alto
Lojas	stores/get/form/register/remove/products/balance/customers, stores/minimum-order, stores/{storeId}/products/{productId}	Onda 2/4	Crítico
Produtos	products/register/remove, products/upload-gallery, products/reorder-gallery, products/import-preview/import, products/{id}/booked-dates	Onda 2	Crítico
Carrinho/Checkout	cart GET/POST/DELETE, checkout/create, checkout/store	Onda 3	Crítico
Pedidos	orders/list, orders/get, orders/register, orders/register-meta, orders/processing, orders/update-delivery-status, orders/customer-	Onda 4	Crítico

Domínio	Endpoints atuais (resumo 1:1)	Prioridade no remake	Risco de quebra
	list, orders/check-payment, order/{grouphash}		
Comentários	comments/list, comments/register	Onda 5	Médio
Entrega	delivery-zipcode/{productId}/{zipcode}, delivery-zipcode/{zipcode}, CEP/cidade por loja/admin	Onda 3/5	Alto
Split e recebedor	info/recipients/status, recipient/register, info/onboarding-profile, info/transfer-settings, sync de recebedor	Onda 4	Crítico
Financeiro	info/financial/overview, info/financial/anticipations*, info/financial/automatic-anticipation-settings, admin/anticipations*	Onda 4/5	Crítico
Admin conteúdo	admin/content*, admin/templates/*, admin/settings/*, admin/abandoned-carts*, admin/recommendations*	Onda 5	Alto
Público conteúdo	content/*, request/products, request/product, request/stores, request/product-interest, request/product-recommendations	Onda 2/4/5	Alto
Taxonomia	group/*, element/*, segmentgroups,	Onda 5	Médio

Domínio	Endpoints atuais (resumo 1:1)	Prioridade no remake	Risco de quebra
	targetadcpbl		
Integrações	hooks/pagarme, hooks/stripe	Onda 4/6	Crítico

27) Ordem de importação e ativação (nível execução)

Para manter o plano limpo, a execução detalhada de importação fica centralizada no item **24**. Aqui ficam só os gates mínimos para operação.

1. Pré-importação validada: snapshot, dry-run e idempotência aprovados.
2. Avanço por onda somente com paridade de dados e rollback pronto.
3. Domínio financeiro só avança com reconciliação consistente de saldo, repasse e antecipação.

28) Análise ampliada de erros e prevenção por domínio

Erros observados + erros prováveis de transição. Aqui está o plano para reduzir risco operacional durante os 90 dias.

28.1 Erros reais já observados (com impacto)

ID	Erro	Sinais reais	Impacto de negócio	Ação no remake
ERR-001	Falha de autorização em integração financeira externa	instabilidade em consulta e criação de antecipações	Admin/lojista sem visão financeira confiável	adaptador resiliente + cache + observabilidade + estado degradado seguro
ERR-002	Falha SMTP (auth e timeout)	15 ocorrências recentes	comunicação transacional falha	fila com retry/backoff + provider fallback + health por provedor
ERR-003	Falha de renovação de token	16 ocorrências recentes	integrações degradadas	token lifecycle manager + alarme antes de expirar
ERR-004	Schema drift (delivery_period)	12 ocorrências	quebra em runtime	migrator gate + verificação de schema no startup
ERR-005	Coluna legado ausente (user em order antigo)	2 ocorrências	falha de criação de pedido em versão intermediária	compat layer de mapeamento + testes de regressão por versão
ERR-006	Campo opcional não protegido (remember_token)	3 ocorrências	login inconsistente	normalização de DTO + nulidade explícita
ERR-007	Permissão/caminho de cache no servidor	permission denied + lock file error	jobs falham e scheduler fica instável	bootstrap de permissões + healthcheck de filesystem
ERR-008	Chunks/manifest 404 no front	incidente real em produção	painéis inacessíveis	deploy atômico + healthcheck pós-build
ERR-009	URL de imagem com domínio incorreto	caso real de upload	produto sem mídia no front	normalizador de URL em persistência e leitura
ERR-010	Lentidão severa em páginas admin	TTFB alto em amostras	queda de produtividade operacional	projeções de leitura + cache de consulta + índices dedicados

28.2 Erros prováveis durante a refatoração (e como prevenir)

ID	Risco provável	Momento	Mitigação prática	Dono
R-001	Divergência de total de pedido entre legado e novo	Onda 4	comparador automático por amostra e bloqueio de release se divergente	Backend
R-002	Vazamento de dados de outra loja no painel lojista	Onda 4	teste de autorização por papel + escopo de loja obrigatório em query	Backend
R-003	Perda de evento de pagamento em concorrência webhook/polling	Onda 4	idempotência por transaction_id + reconciliador contínuo	Backend
R-004	Quebra de mobile em páginas longas de painel	Ondas 4-5	smoke visual obrigatório em viewport mobile	Frontend
R-005	Falha em importação de mídia (arquivo órfão)	Onda 2	scanner de mídia pós-lote + reprocessamento	Dados
R-006	Acúmulo de jobs e atraso de reconciliação	Ondas 4-5	fila com prioridade + limite por worker + alerta de atraso	Ops
R-007	Regressão de SEO em páginas públicas	Onda 2	snapshot de metadados antes/depois + fallback de renderização	Frontend

28.3 Matriz gatilho-resposta (erro → ação automática)

Gatilho	Ação automática imediata	Ação operacional em seguida	SLA interno
Erro financeiro 5xx contínuo > 3 minutos	ativar modo degradado (somente leitura financeira)	abrir incidente + coleta de evidência técnica para suporte	15 minutos
Scheduler crítico sem execução por > 10 minutos	forçar execução imediata + abrir incidente S1	validar filas e último run de <code>orders:reconcile</code> , <code>financial:reconcile-balances</code> e <code>ops:monitor-health</code>	10 minutos
Falha webhook pagamento	enfileirar reconciliação forçada por pedido	validar status no gateway e atualizar timeline do pedido	5 minutos
Divergência de total detectada	bloquear fluxo de confirmação automática	triagem manual e correção de regra	imediato
Erro de autorização em painel	revogar sessão e bloquear rota de risco	auditoria de escopo + hotfix	imediato
Erro de mídia em upload	gravar evento em fila de reparo	reprocessar arquivo e normalizar URL	30 minutos

29) Checklist de aceite final por feature crítica (antes de cada avanço de onda)

Este checklist é o que impede “passar bug crítico de fase”. Sem cumprir, a onda não avança.

29.1 Fluxo público e checkout

1. Produto carrega mídia principal e galeria sem URL quebrada.
2. Seleção de variação/adicional respeita min/max e reflete no preço.
3. Carrinho mantém itens após refresh e após login.
4. Frete calcula com timeout controlado e fallback amigável.
5. Checkout cria pedido com subtotal/frete/total consistentes.
6. Página pós-pagamento reflete estado real sem depender só de webhook.

29.2 Fluxo de pedidos (cliente, lojista e admin)

1. Cliente só vê dados do próprio pedido.
2. Lojista só vê dados dos itens da própria loja.
3. Admin vê visão completa consolidada com histórico de eventos.
4. Timeline não duplica eventos com semântica equivalente.
5. Método de pagamento aparece corretamente nos painéis.
6. Status de entrega atualiza com rastreabilidade.

29.3 Fluxo financeiro e antecipações

1. Saldo disponível, aguardando e transferido com fonte explícita.
2. Antecipação manual cria registro e retorna estado previsível.
3. Falha de consulta no gateway mostra estado degradado sem travar painel.
4. Configuração automática salva e reflete em leitura subsequente.
5. Conciliação financeira roda sem intervenção de admin não técnico.

29.4 Operação e manutenção

1. Deploy não conclui sem healthcheck backend+frontend válido.
2. Scheduler obrigatório ativo: `orders:reconcile`, `financial:reconcile-balances`, `ops:monitor-health`.
3. Backup diário (MySQL + RavenDB) com retenção, dono definido e teste de restauração semanal com evidência.
4. Logs críticos com alerta e classificação por severidade.
5. Rollback testado no mesmo ambiente antes de produção.

Nota de condução: os pontos mais sensíveis de RavenDB e .NET passam por checkpoint externo pontual, mas sem travar a execução do plano.

30) Análises específicas e melhorias priorizadas para produção

Esta seção consolida diagnóstico prático de operação e lista melhorias de maior retorno para estabilidade, receita e previsibilidade de suporte.

30.1 Diagnóstico específico (pontos com impacto real)

Domínio	Sinal observado	Risco de negócio	Direção de correção
Financeiro (antecipações)	Intermitência em operações de consulta/criação no gateway externo	Painel admin/lojista sem visão financeira confiável em alguns momentos	Adapter resiliente com cache curto, retry controlado e modo degradado explícito
Comunicação transacional	Falhas de autenticação/timeout no canal de e-mail	Notificações críticas de pedido podem não chegar	Revalidar credenciais, healthcheck ativo e fallback operacional por canal secundário
Painéis admin	Picos de latência (TTFB alto) em páginas sensíveis	Operação lenta e aumento de tempo de resposta do suporte	Projeções de leitura, índices dedicados e redução de consultas pesadas por página
Observabilidade e logs	Crescimento acelerado de logs e excesso de ruído operacional	Dificulta triagem e aumenta custo de disco	Rotação de logs, retenção por criticidade e padronização de eventos de erro

30.2 Melhorias priorizadas (impacto x esforço)

Prioridade	Melhoria	Impacto	Esforço	Critério de aceite
P0	Resiliência financeira no fluxo de antecipação	Alta	Médio	Painéis financeiros não travam quando integração externa oscila
P0	Correção do canal transacional de e-mail (auth + timeout + fallback)	Alta	Baixo/Médio	Notificações de pedido com taxa de envio estável e monitorada
P0	Política de rotação e retenção de logs de API e containers	Alta	Baixo	Sem crescimento descontrolado de log e com histórico útil para suporte
P1	Otimização de consultas dos painéis admin mais acessados	Alta	Médio	Queda perceptível de TTFB nas telas críticas de operação
P1	Fortalecimento de monitor de scheduler e jobs críticos	Média/Alta	Baixo	Alerta antecipado quando job crítico atrasa ou para
P2	Redução de ruído de bots/scanners na borda	Média	Baixo	Menos 404/400 de varredura interferindo na leitura operacional

30.3 Sequência recomendada de execução

1. Fechar P0 de e-mail e rotação de logs para estabilizar operação diária.
2. Aplicar P0 de resiliência financeira para reduzir atrito em admin/lojista.
3. Executar P1 de performance nas páginas administrativas com maior tráfego.
4. Ativar reforço de monitoramento de scheduler para resposta antecipada.
5. Refinar P2 de ruído externo para melhorar sinal de observabilidade.

30.4 Critérios de aprovação desta frente

1. Painéis críticos operando sem bloqueio em cenário degradado de integração.
2. Notificações transacionais com taxa estável e visibilidade de falha.
3. Redução de latência nas telas admin priorizadas.
4. Ambiente com logs sob controle e sem crescimento anormal.

Fechamento do plano

Eu considero esse plano viável em 3 meses, desde que a gente respeite os gates e não abra escopo fora da rota. A base está pronta: inventário fechado, ordem de migração clara, riscos mapeados e plano de rollback completo. A prioridade é

simples: migrar com segurança, manter receita estável e encerrar o ciclo com backend escalável para a próxima fase.

Documento interno de planejamento técnico. Autor: Pedro (Tech Lead).
Material preparado para validação técnica pontual sem bloquear início da execução.